LECTURE 19

Logistic Regression II

Making decisions and assessing performance.

Presented by Dr . Devan Becker, Data200

Adapted from Data 100, Summer 2023 @ UC Berkeley by Bella Crouch and Dominic Liu

Content credit: Acknowledgments

CC BY NC SA

Goals for this Lecture

Digging more deeply into logistic regression

- Applying decision rules to make a classification
- Learning when logistic regression works well, and when it does not
- Introducing new metrics for model performance

CC BY NC SA

Agenda

- Decision Boundaries
- Linear Separability
- Performance Metrics
- Adjusting the Classification Threshold

CC BY NC SA

Decision Boundaries

- **Decision Boundaries**
- Linear Separability
- Performance Metrics
- Adjusting the Classification Threshold

BY NC SA

Review: Logistic Regression Process

In logistic regression, we model the *probability* that a datapoint belongs to Class 1.

bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
39.1	18.7	181.0	3750.0
39.5	17.4	186.0	3800.0
40.3	18.0	195.0	3250.0
NaN	NaN	NaN	NaN
36.7	19.3	193.0	3450.0

Input: numeric features

$$p = \sigma\left(x^\top \theta\right)$$

Model: linear combination transformed by **sigmoid**

IsAdelie?
If y_hat > 0.5: predict "Adelie"
Other: predict "not Adelie"

Decision rule

Output: **class**

Yesterday

Today

CC BY NC SA

Applying a Decision Rule

Last time, we developed the logistic regression model to predict the probability of a datapoint belonging to Class 1. We never actually made any *classifications* (ie deciding to classify a point into Class 0 or Class 1).

$$p = P(Y = 1 \mid x) = \frac{1}{1 + e^{-x^T \theta}}$$

A **decision rule** tells us how to interpret the output of the model to make a decision on how to classify a datapoint.

CC BY NC SA

Thresholds

We commonly make decision rules by specifying a **threshold**, T. If the predicted probability is greater than T, predict Class 1. Otherwise, predict Class 0.

$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

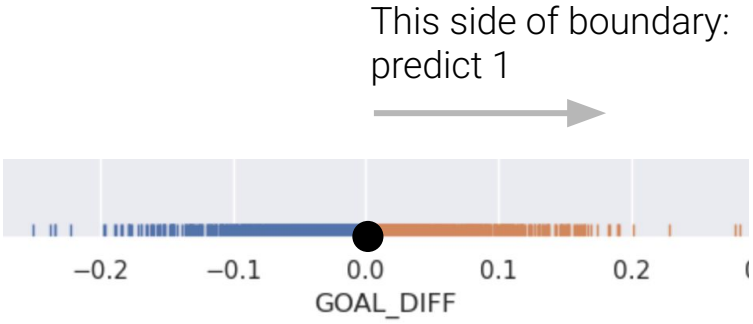
The threshold is often 0.5, but *not always*. We'll discuss why we might use $T \neq 0.5$ later this lecture.

CC BY NC SA

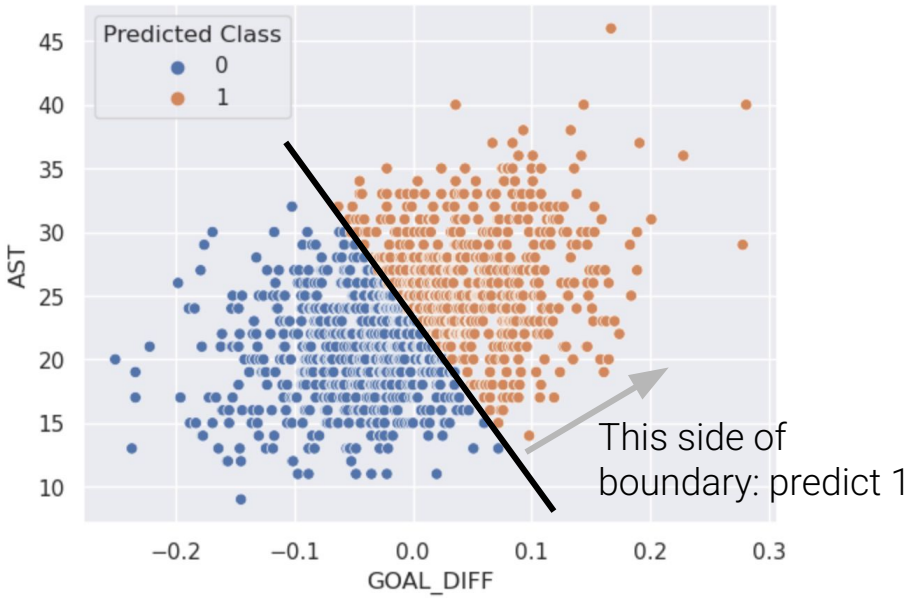
Decision Boundaries

A **decision boundary** describes the "line" the splits the data into classes based on its **features**.

- For logistic regression, the decision boundary is a **hyperplane**: a linear combination of the features in p-dimensions



1 feature: decision boundary is a 1D point



2 features: decision boundary is a 2D line

CC BY NC SA

Decision Boundaries

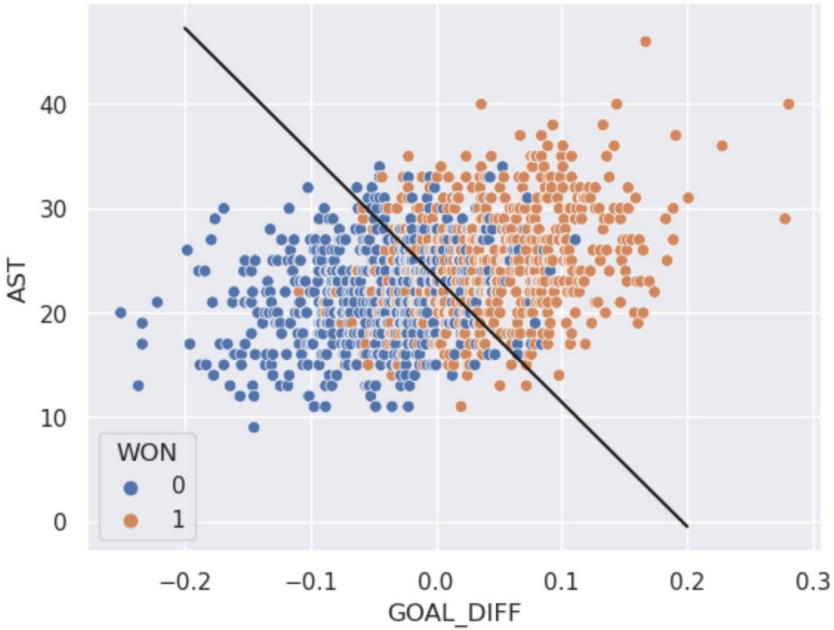
We can recover the decision boundary from the final logistic regression model.

Example: model with two features

$$T = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \text{GOAL_DIFF} + \theta_2 \text{AST})}}$$

$$\theta_0 + \theta_1 \text{GOAL_DIFF} + \theta_2 \text{AST} = -\log\left(\frac{1}{T} - 1\right)$$

The decision boundary is a line in terms of the features.



Here, the color is the *true* class, rather than the model's predictions.

CC BY NC SA

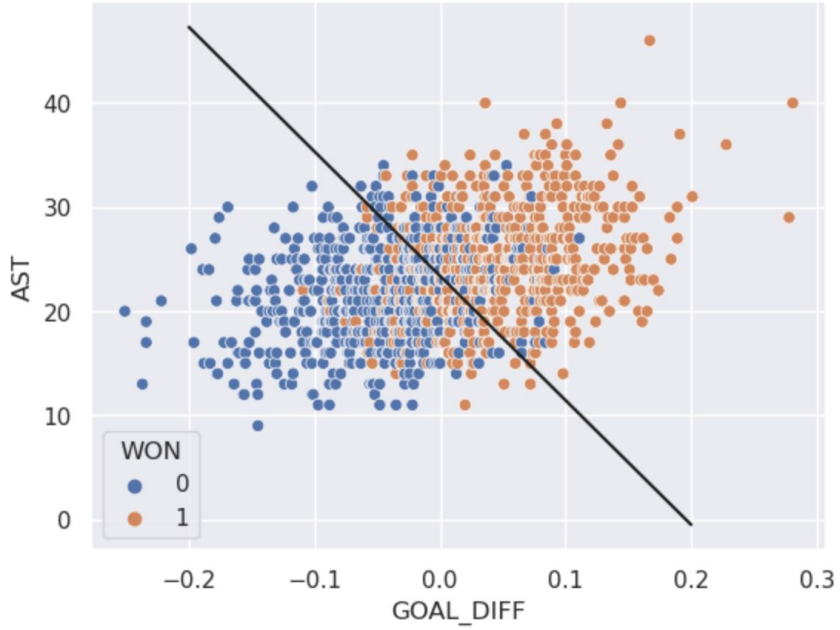
Linear Separability

- Decision Boundaries
- **Linear Separability**
- Cross-Entropy Loss
- Performance Metrics
- Adjusting the Classification Threshold

CC BY NC SA

Decision Boundaries

Our decision boundary in the previous example did not perfectly separate the two classes.



There's a "muddled" region near the decision boundary where our classifier predicts the wrong class.

What would the data have to look like for the classifier to make perfect predictions?

CC BY NC SA

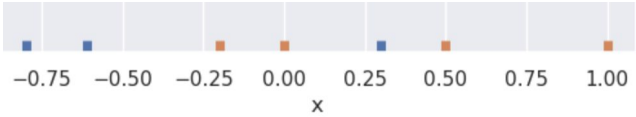
Linear Separability

A classification dataset is said to be **linearly separable** if there exists a hyperplane **among input features x** that separates the two classes y.

If there is one feature, look for a point that separates the classes.

separable

not separable



CC BY NC SA

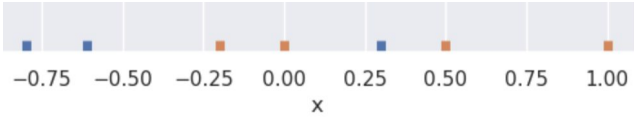
Linear Separability

A classification dataset is said to be **linearly separable** if there exists a hyperplane **among input features x** that separates the two classes y.

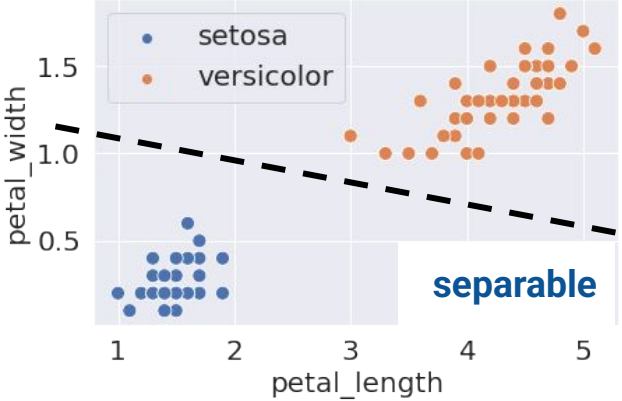
If there is one feature, look for a point that separates the classes.

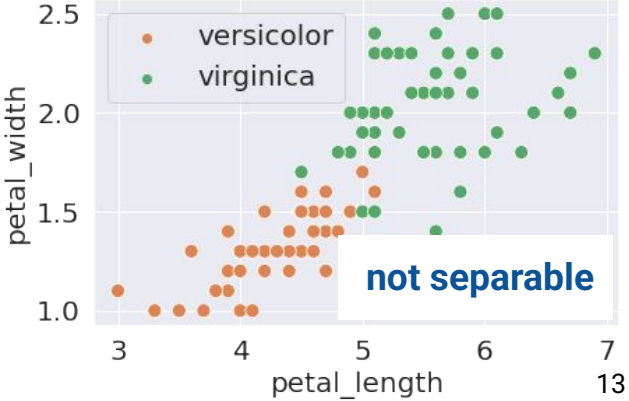
separable

not separable



If there are two features, look for a line that separates the classes





CC BY NC SA

Linear Separability and Loss

When the dataset is linearly separable, a logistic regression classifier can perfectly assign datapoints into classes.

Can it achieve 0 cross-entropy loss?

$$-(y\log(p) + (1-y)\log(1-p))$$

Loss is 0 if:

- p = 1 when y = 1
- p = 0 when y = 0

CC BY NC SA

Linear Separability and Loss

When the dataset is linearly separable, a logistic regression classifier can perfectly assign datapoints into classes.

Can it achieve 0 cross-entropy loss?

$$-\left(y\log(p) + (1-y)\log(1-p)\right)$$

Loss is 0 if:

- p = 1 when y = 1
- p = 0 when y = 0

Consider a simple model with one feature and no intercept.

$$\hat{P}_\theta(Y=1|x) = \sigma(\theta x) = \frac{1}{1+e^{-\theta x}}$$

What θ will achieve 0 loss if we train on the datapoint x = 1, y = 1?

CC BY NC SA

What value of theta leads to 0 cross-entropy loss on this point?

BY NC SA

Linear Separability and Loss

When the dataset is linearly separable, a logistic regression classifier can perfectly assign datapoints into classes.

Can it achieve 0 cross-entropy loss?

$$-\left(y\log(p) + (1-y)\log(1-p)\right)$$

Loss is 0 if:

- p = 1 when y = 1
- p = 0 when y = 0

Consider a simple model with one feature and no intercept.

$$\hat{P}_\theta(Y=1|x) = \sigma(\theta x) = \frac{1}{1+e^{-\theta x}}$$

What θ will achieve 0 loss if we train on the datapoint x = 1, y = 1?

Want p = 1.
Occurs as $\theta \to \infty$

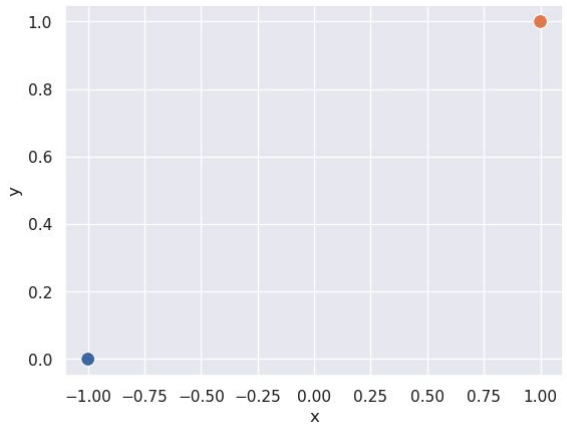
CC BY NC SA

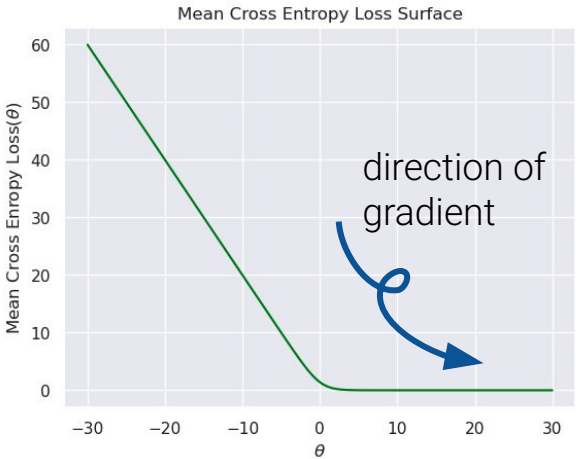
Linear Separability Causes Diverging Model Parameters

When data is linearly separable, the optimal model parameters **diverge** to ±∞

- The sigmoid can never output exactly 0 or 1, so no finite optimal theta exists

This can be a problem when using gradient descent to fit the model. Consider a simple, linearly separable "toy" dataset with two datapoints.





(Impossible to see, but) plateau is slightly tilted downwards.

Loss approaches 0 as theta increases.

Because gradient descent follows the tilted loss surface downwards, it never converges.

CC BY NC SA

Linear Separability Causes Diverging Model Parameters

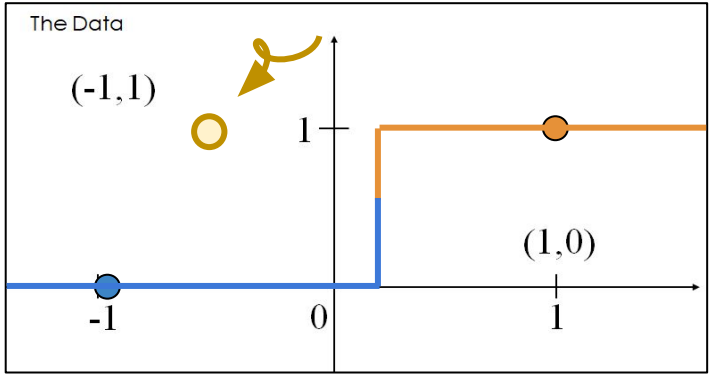
A model with infinite parameter values is **overconfident**.

- Consider a new point (-0.5, 1).
- The model incorrectly says **p = 0**, so it predicts **0**.

$$-\left(y\log(p) + (1-y)\log(1-p)\right)$$

$$\rightarrow 1\log(0)$$

Loss is infinite.



How to avoid model parameters getting too large? Regularize!

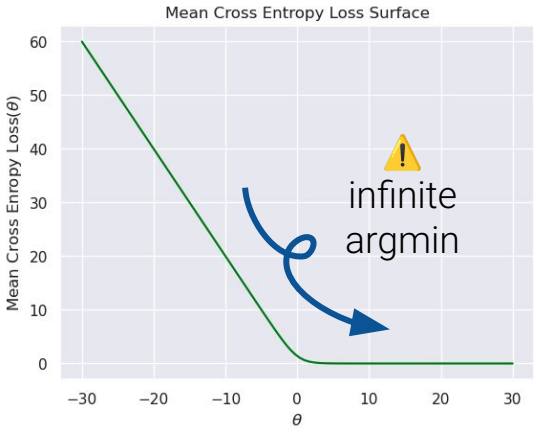
CC BY NC SA

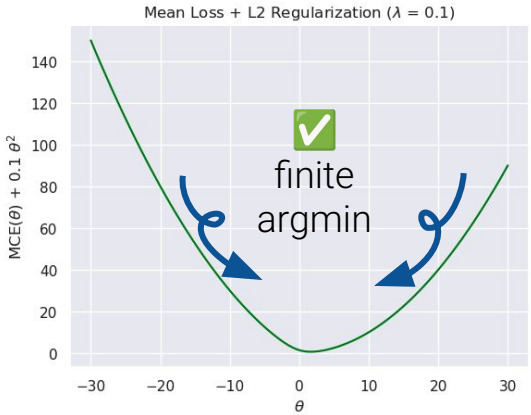
Regularized Logistic Regression

To avoid large weights (particularly on linearly separable data), use **regularization**.

- As with linear regression, standardize features first.

$$\underset{\theta}{\operatorname{argmin}} -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\sigma(X_i^T\theta) + (1-y_i)\log(1-\sigma(X_i^T\theta))\right) + \lambda \sum_{j=1}^{d}\theta_j^2$$






```
# sklearn defaults
model = LogisticRegression(
          penalty='l2', C=1.0, …)
model.fit()
```

Regularization hyperparameter C is the inverse of λ. C = 1 / λ.

Set C big for minimal regularization, e.g., `C=300.0`.

CC BY NC SA

An Aside

In Data 200, we will only use `sklearn` to fit logistic regression models.

There is no closed-form solution to the optimal theta vector.

Why not gradient descent? The gradient is a little messy (see bonus slides for details)

CC BY NC SA

Performance Metrics

- Decision Boundaries
- Linear Separability
- **Performance Metrics**
- Adjusting the Classification Threshold

CC BY NC SA

Modeling Process

	Regression ($y \in \mathbb{R}$)	Classification ($y \in \{0, 1\}$)
1. Choose a model ✅	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	Logistic Regression $\hat{P}_\theta(Y = 1\|x) = \sigma(x^T\theta)$
2. Choose a loss function ✅	Squared Loss or Absolute Loss	Average Cross-Entropy Loss $-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\sigma(X_i^T\theta) + (1 - y_i)\log(1 - \sigma(X_i^T\theta))\right)$
3. Fit the model	Regularization Sklearn/Gradient descent	`sklearn` with regularization
4. Evaluate model performance	R^2, Residuals, etc.	Let's do it!

CC BY NC SA

Why More Performance Metrics?

We've already introduced cross-entropy loss – why do we need additional ways of assessing how well our models perform?

- In linear regression, we made numerical predictions and used a loss function to determine how "good" these predictions were
- In logistic regression, our ultimate goal is to *classify* data – we are more concerned with whether or not each datapoint was assigned the correct class using the decision rule

The performance metrics we are about to introduce will assess the quality of classifications, not the predicted probabilities.

CC BY NC SA

Classifier Accuracy

Now that we actually have our classifier, let's try and quantify how well it performs.

The most basic evaluation metric for a classifier is **accuracy**.

$$\text{accuracy} = \frac{\#\text{ of points classified correctly}}{\#\text{ points total}}$$

```
def accuracy(X, Y):
    return np.mean(model.predict(X) == Y)

accuracy(X, Y) # 0.794
```

```
model.score(X, Y) # 0.794
```

(sklearn documentation)

While widely used, the accuracy metric is **not so meaningful** when dealing with **class imbalance**.

CC BY NC SA

Pitfalls of Accuracy: A Case Study

Suppose we're trying to build a classifier to filter spam emails (Project B!).

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are truly **ham**.

Your friend ("Friend 1"):

Classify *every* email as **ham** (0).

$$\hat{y} = \text{classify}_{\text{friend}}(x) = 0$$

1. What is the accuracy of your friend's classifier?
2. Is accuracy a good metric of this classifier's performance?

CC BY NC SA

We have 100 emails, of which only 5 are truly spam, and the remaining 95 are ham. Assume that we have a model that classifies all emails as ham. What is the accuracy of the classifier?

CC BY NC SA

Pitfalls of Accuracy: A Case Study

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Your friend ("Friend 1"):

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

High accuracy...
...but we detected **none** ⚠️ of the spam!!!

CC BY NC SA

Pitfalls of Accuracy: A Case Study

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Your friend ("Friend 1"):

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

High accuracy...
...but we detected **none** ⚠️ of the spam!!!

Your other friend ("Friend 2"):

Classify every email as **spam** (1).

$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

Low ⚠️ accuracy...
...but we detected **all** of the spam!!!

CC BY NC SA

Pitfalls of Accuracy: Class Imbalance

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Accuracy is not always a good metric for classification, particularly when your data have **class imbalance** (e.g., very few 1's compared to 0's).

Your friend ("Friend 1"):

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

High accuracy...
...but we detected **none** ⚠️ of the spam!!!

Your other friend:

Classify every email as **spam** (1).

$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

Low ⚠️ accuracy...
...but we detected **all** of the spam!!!

CC BY NC SA

Types of Classifications

There are four different classifications that our model might make:

- True positive: correctly classify a positive point as being positive ($y = 1, \hat{y} = 1$)
- True negative: correctly classify a negative point as being negative ($y = 0, \hat{y} = 0$)
- False positive: incorrectly classify a negative point as being positive ($y = 0, \hat{y} = 1$)
- False negative: incorrectly classify a positive point as being negative ($y = 1, \hat{y} = 0$)

A confusion matrix plots these quantities for a particular classifier and dataset.

"**positive**" means a prediction of **1**.
"**negative**" means a prediction of **0**.

Prediction $\hat{y}$

Actual y	**0**	**1**
0	True **negative** (TN)	False **positive** (FP)
1	False **negative** (FN)	True **positive** (TP)

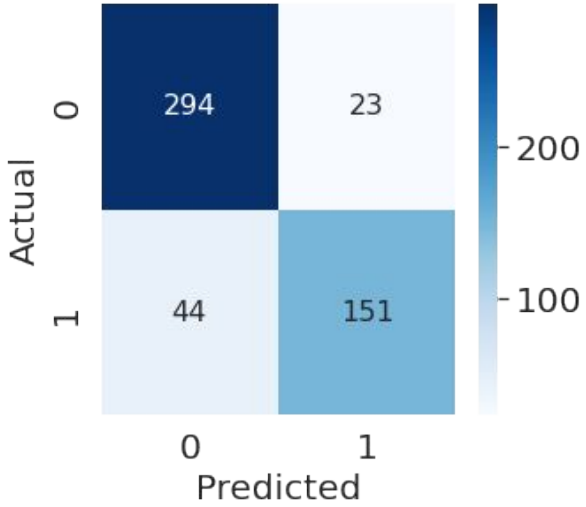
CC BY NC SA

Types of Classifications

A confusion matrix plots these quantities for a particular classifier and dataset.

In `sklearn`:

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_true, Y_pred)
```



CC BY NC SA

Accuracy, Precision, and Recall

$$\text{accuracy} = \frac{TP + TN}{n}$$

What proportion of points did our classifier classify correctly?

		Prediction	
		0	**1**
Actual	0	TN	FP
	1	FN	TP

CC BY NC SA

Accuracy, Precision, and Recall

$$\text{accuracy} = \frac{TP + TN}{n}$$

What proportion of points did our classifier classify correctly?

		Prediction	
		0	**1**
Actual	0	TN	FP
	1	FN	TP

Precision and recall are two commonly used metrics that measure performance even in the presence of class imbalance.

$$\text{precision} = \frac{TP}{TP + FP}$$

Of all observations that were predicted to be 1, what proportion were actually 1?

- How **precise** is our classifier **when it is positive**?
- Penalizes false positives.

CC BY NC SA

Accuracy, Precision, and Recall

$$\text{accuracy} = \frac{TP + TN}{n}$$

What proportion of points did our classifier classify correctly?

		Prediction	
		0	**1**
Actual	0	TN	FP
	1	FN	TP

Precision and **recall** are two commonly used metrics that measure performance even in the presence of class imbalance.

$$\text{precision} = \frac{TP}{TP + FP}$$

Of all observations that were predicted to be 1, what proportion were actually 1?

- How accurate is our classifier when it is positive?
- Penalizes false positives.

$$\text{recall} = \frac{TP}{TP + FN}$$

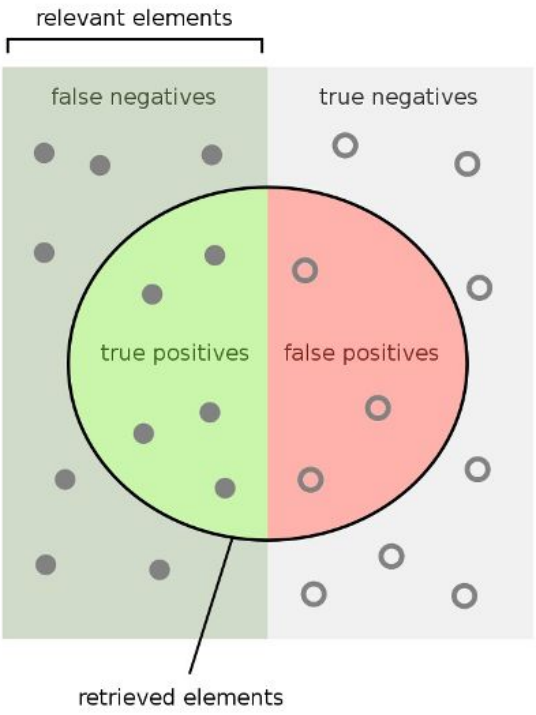
Of all observations that were actually 1, what proportion did we predict to be 1? (Also known as sensitivity.)

- How **sensitive** is our classifier to **positives**?
- Penalizes false negatives.

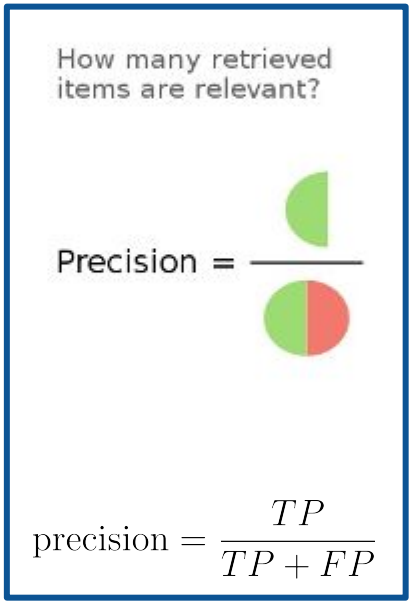
CC BY NC SA

One of the Most Valuable Graphics on Wikipedia

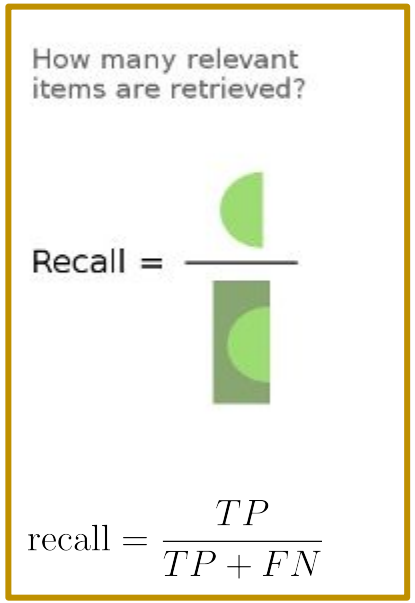
(i.e., positive;
predicted class is 1)



(*i.e., true class is 1)



$$\text{precision} = \frac{TP}{TP + FP}$$



$$\text{recall} = \frac{TP}{TP + FN}$$

[adapted from Wikipedia]

CC BY NC SA

Back to the Spam

$$\text{accuracy} = \frac{TP + TN}{n}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Your friend:

Classify every email as **ham** (0).

	0	**1**
0	TN: 95	FP: 0
1	FN: 5	TP: 0

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

$$\text{precision}_1 = \frac{0}{0 + 0} = \text{undefined}$$

$$\text{recall}_1 = \frac{0}{0 + 5} = 0$$

CC BY NC SA

Back to the Spam

$$\text{accuracy} = \frac{TP + TN}{n}$$
$$\text{precision} = \frac{TP}{TP + FP}$$
$$\text{recall} = \frac{TP}{TP + FN}$$

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Your friend:

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

Never positive!

$$\text{precision}_1 = \frac{0}{0+0} = \text{undefined}$$

$$\text{recall}_1 = \frac{0}{0+5} = 0$$

Your other friend ("Friend 2"):

Classify every email as **spam** (1).

	0	**1**
0	TN: 0	FP: 95
1	FN: 0	TP: 5

$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

$$\text{precision}_2 = \frac{5}{5+95} = 0.05$$

Many false positives!

$$\text{recall}_2 = \frac{5}{5+0} = 1.0$$

No false negatives!

CC BY NC SA

Precision vs. Recall

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

Precision penalizes false positives, and Recall penalizes false negatives.

This suggests that there is a **tradeoff** between precision and recall; they are often inversely related.

- Ideally, both would be near 100%, but that's unlikely to happen.

CC BY NC SA

Which Performance Metric?

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

Precision penalizes false positives, and Recall penalizes false negatives.

In many settings, there might be a higher “cost” to missing positive or negative cases.

Some examples:

- Detecting if someone tests positive (1) or negative (0) for a disease
- Determining if someone should be sentenced to prison (1) or not (0)
- Filtering an email as spam (1) or ham (0)

CC BY NC SA

Which performance metric?

BY NC SA

True and False Positive Rates

Keeping things interesting – two more performance metrics.

$$\text{FPR} = \frac{FP}{FP + TN}$$

False Positive Rate (FPR): out of all datapoints that had Y=0, how many did we classify **incorrectly**?

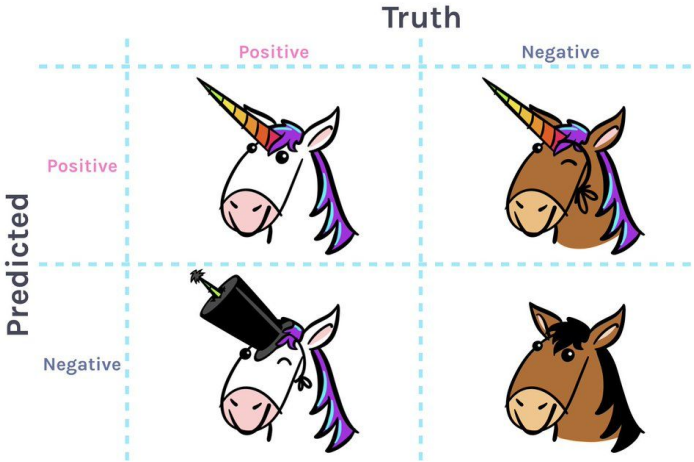
$$\text{TPR} = \frac{TP}{TP + FN}$$

True Positive Rate (TPR): out of all datapoints that had Y=1, how many did we classify correctly? Same as recall.

		Prediction	
		0	**1**
Actual	0	TN	FP
	1	FN	TP

CC BY NC SA

Interlude



CC BY NC SA

Adjusting the Classification Threshold

- Decision Boundaries
- Linear Separability
- Performance Metrics
- **Adjusting the Classification Threshold**

CC BY NC SA

Thresholds

We commonly make decision rules by specifying a **threshold**, T. If the predicted probability is greater than T, predict Class 1. Otherwise, predict Class 0.

$$p = P(Y = 1 \mid x) = \frac{1}{1 + e^{-x^T \theta}}$$

$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

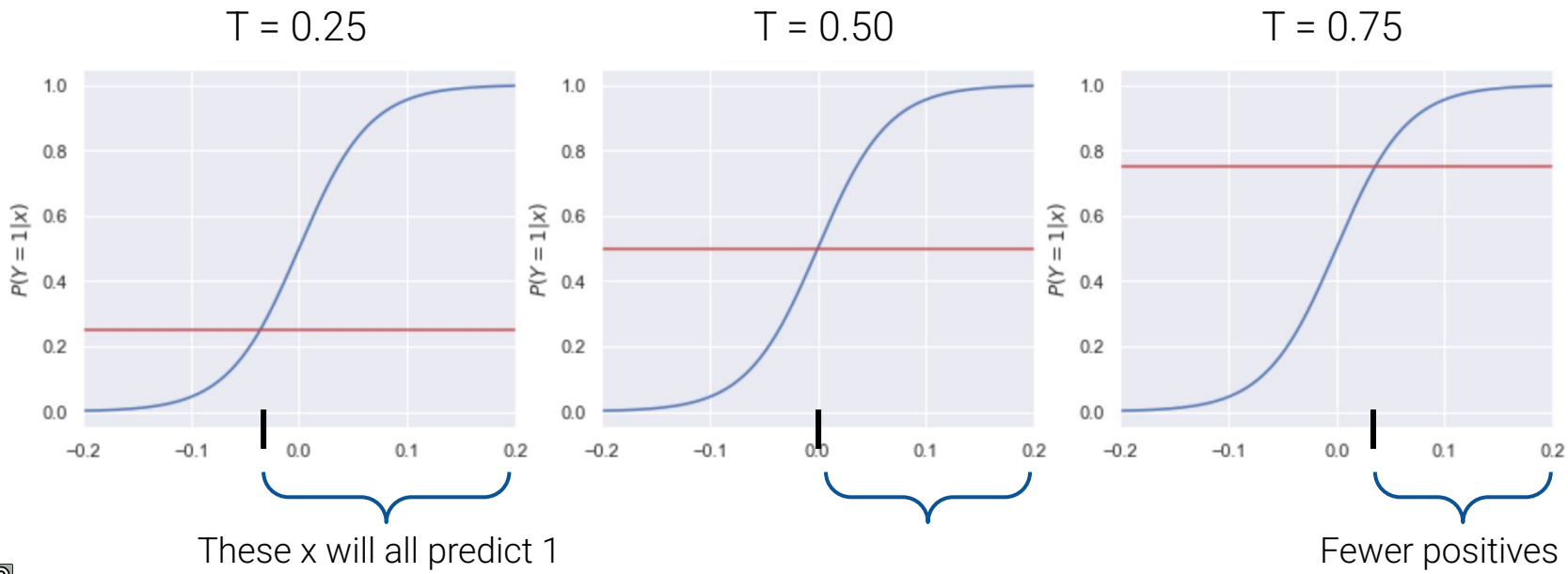
What happens if we set T to something other than 0.5?

CC BY NC SA

Changing the Threshold

$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

As we increase the threshold T, we "raise the standard" of how confident our classifier needs to be to predict 1 (i.e., "positive").



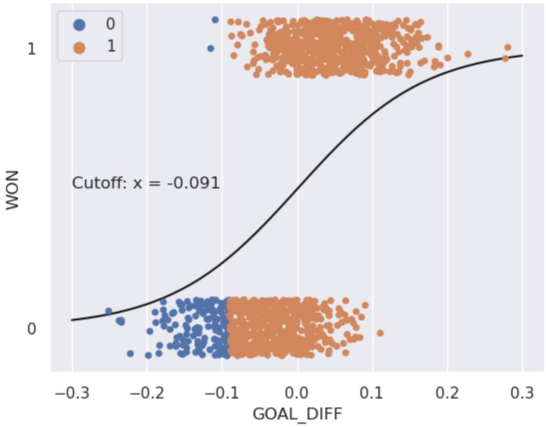
CC BY NC SA

Changing the Threshold

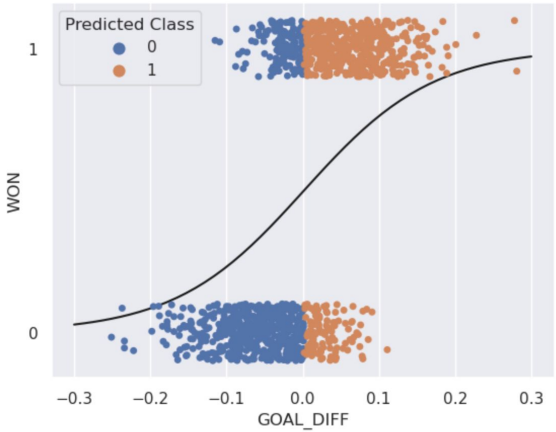
$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

As we increase the threshold T, we “raise the standard” of how confident our classifier needs to be to predict 1 (i.e., “positive”).

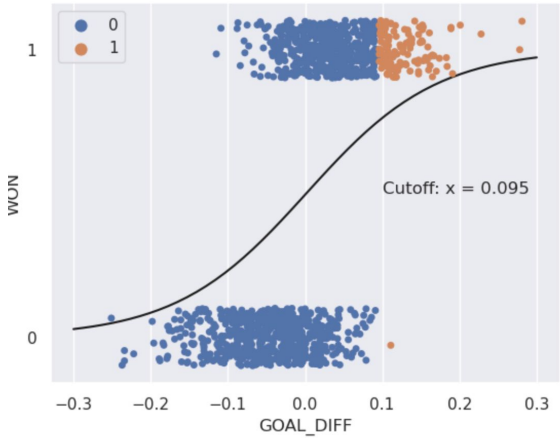
T = 0.25



T = 0.50



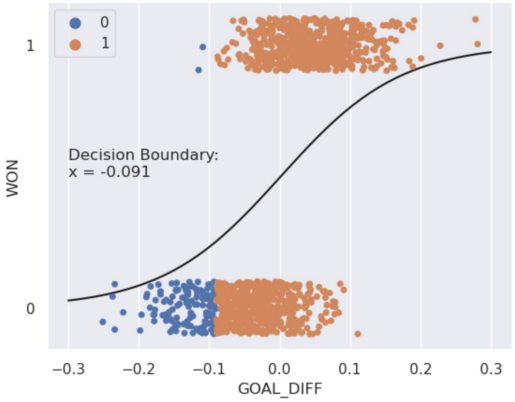
T = 0.75



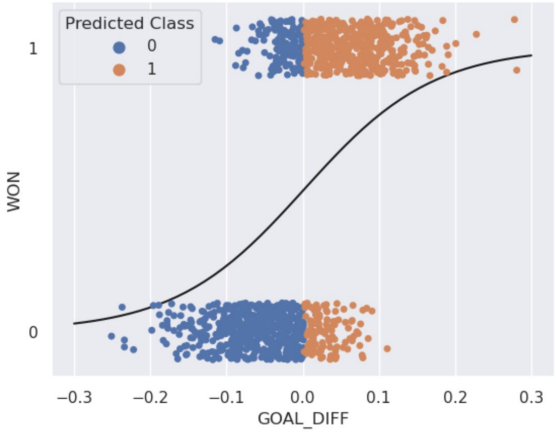
CC BY NC SA

Changing the Threshold

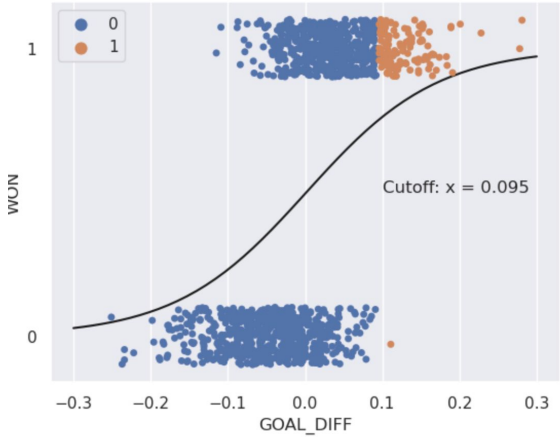
T = 0.25



T = 0.50



T = 0.75



How to interpret a higher classification threshold: the model needs to predict a higher probability of a point belonging to Class 1 before it can confidently classify it as Class 1

- As T increases, we predict fewer positives!

Changing the threshold allows us to finetune how “confident” we want our model to be before making a positive prediction

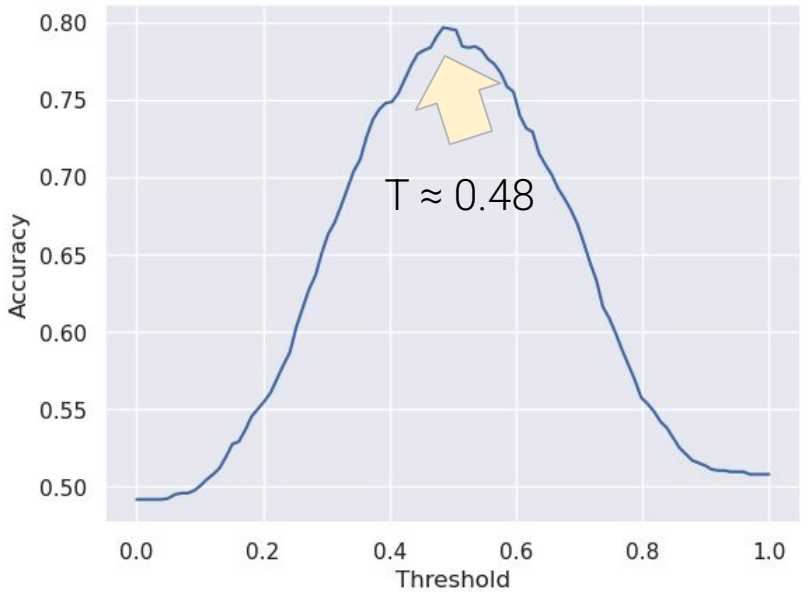
CC BY NC SA

Choosing an Accuracy Threshold

The choice of threshold T impacts classification performance.

- High T: Most predictions are 0. Lots of false negatives.
- Low T: Most predictions are 1. Lots of false positives.

Do we get max accuracy when T = 0.5? Not always the case…



Demo

See notebook for code snippets.

CC BY NC SA

Precision-Recall Curves

Earlier, we noticed that there is a tradeoff between precision (penalizes false positives) and recall (penalizes false negatives).

We should choose a threshold that keeps both precision and recall high.

In a precision-recall curve, we:

1) Test out many different possible thresholds
2) For each threshold, compute the precision and recall of the classifier
3) Choose the threshold at the “corner” of the curve

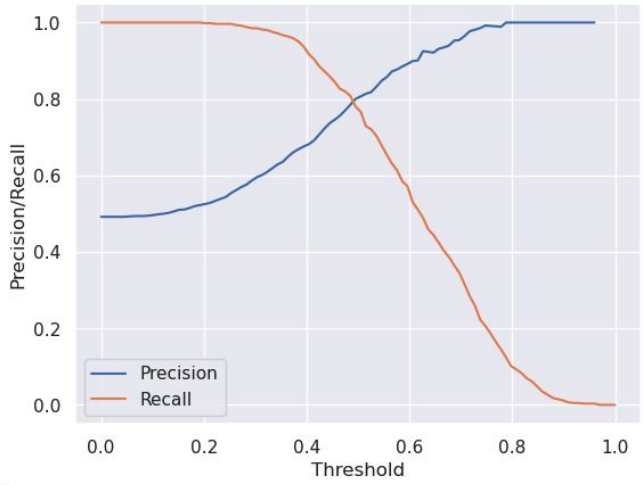
Demo

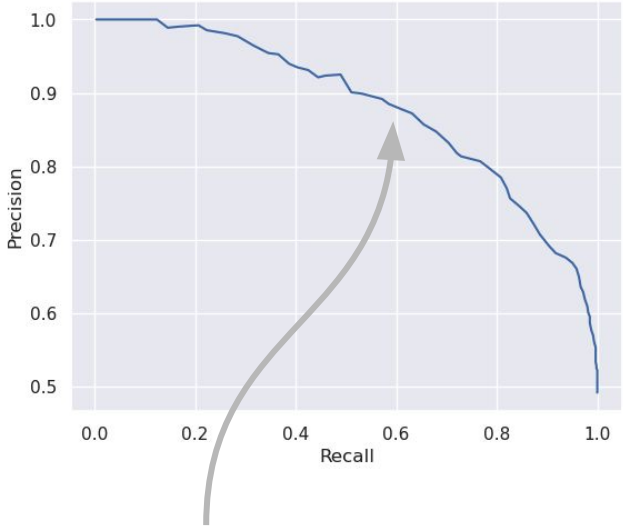
CC BY NC SA

Precision-Recall Curves

In a precision-recall curve, we:

1) Test out many different possible thresholds
2) For each threshold, compute the precision and recall of the classifier





We should choose a threshold that keeps both precision and recall high.

Demo

CC BY NC SA

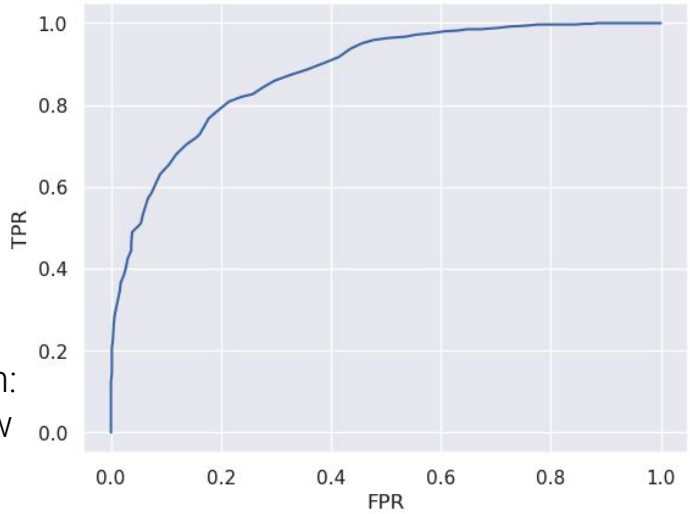
ROC Curves

We can perform a similar process with FPR and TPR

1) Try many thresholds
2) Compute the FPR and TPR for each threshold
3) Choose a threshold that keeps FPR *low* and TPR *high*

ROC = “receiver operating characteristic”, comes from radar in WWII

Threshold is low: FPR high, TPR high (many positive predictions)



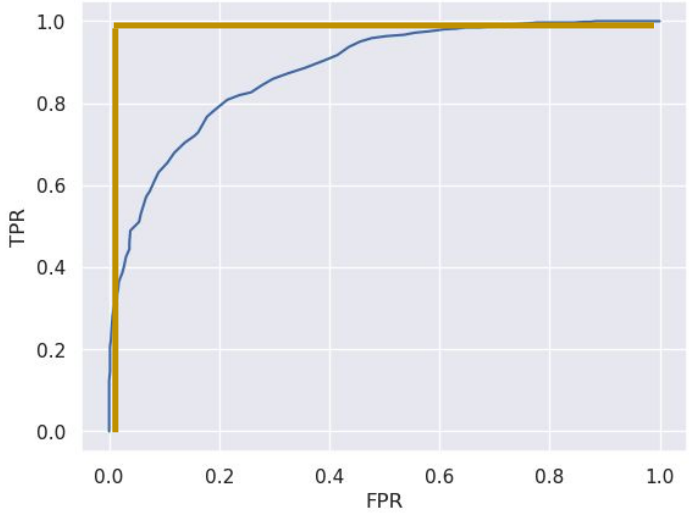
Threshold is high: FPR low, TPR low (few positive predictions)

Demo

CC BY NC SA

ROC Curves

A **perfect predictor** has TPR = 1 and FPR = 0



The **Area Under Curve (AUC)** of the perfect predictor is 1.

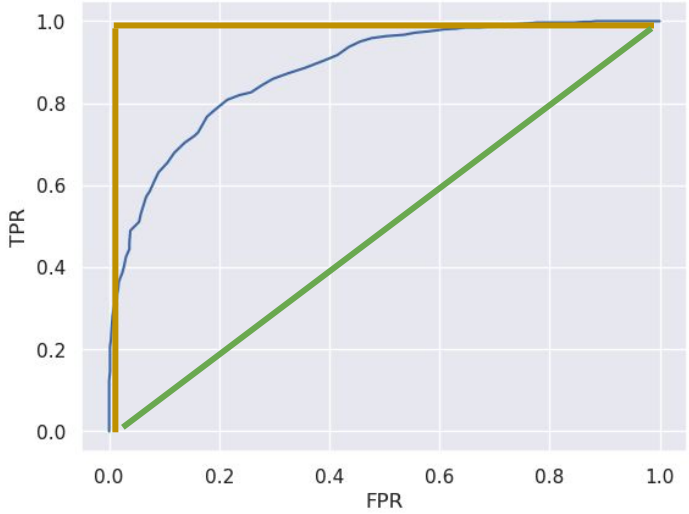
Because we want our classifier to be as close as possible to the perfect predictor, we aim to maximize the AUC.

Demo

CC BY NC SA

ROC Curves

A predictor that **predicts randomly** has an AUC of 0.5



See bonus slides for proof.

Real-world classifiers have an AUC between 0.5 and 1.

Demo

CC BY NC SA

Participation Questions

BY NC SA

Q1

Logistic Regression results in a linear decision boundary.

A. True

B. False

CC BY NC SA

Q2

Which of the following will *increase* the false positive rate for *training* data?

A. Changing the decision threshold to something other than 0.5

B. Fitting a more complex model

C. Regularizing the parameters

CC BY NC SA

Q3

How many different thresholds are shown in a ROC plot?

A. 0

B. 1

C. Infinite

CC BY NC SA

Q4

AndI

A. Values of the features

B. Probability of being in class 0

C. Probability of being in class 1

D. Value of the target

CC BY NC SA

Q5

What should we optimize?

A. All of the below, with varying importance depending on context

B. Precision

C. Recall

D. True Positive Rate

E. True Negative Rate

F. Accuracy

G. ...

CC BY NC SA

[Extra] Gradient Descent for Logistic Regression

- Decision Boundaries
- Linear Separability
- Performance Metrics
- Adjusting the Classification Threshold

CC BY NC SA

[Extra] Gradient Descent for Logistic Regression

	Regression ($y \in \mathbb{R}$)	Classification ($y \in \{0, 1\}$)
1. Choose a model ✅	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	Logistic Regression $\hat{P}_\theta(Y = 1\|x) = \sigma(x^T\theta)$
2. Choose a loss function ✅	Squared Loss or Absolute Loss	Average Cross-Entropy Loss $-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\sigma(X_i^T\theta) + (1 - y_i)\log(1 - \sigma(X_i^T\theta))\right)$
3. Fit the model	Regularization Sklearn/Gradient descent	Regularization Sklearn/Gradient descent
4. Evaluate model performance ✅	R^2, Residuals, etc.	Accuracy, Precision, Recall, ROC Curves

CC BY NC SA

A Simplification

$$y_i \log(p_i) + (1 - y_i)\log(1 - p_i)$$

$$= y_i \log\left(\frac{p_i}{1 - p_i}\right) + \log(1 - p_i)$$

$$= y_i \phi(x_i)^T \theta + \log(\sigma(-\phi(x_i)^T \theta))$$

The calculation on the left uses the following:

$$t_i = \phi(x_i)^T \theta$$

$$p_i = \sigma(t_i)$$

$$t_i = \log\left(\frac{p_i}{1 - p_i}\right)$$

$$1 - \sigma(t_i) = \sigma(-t_i)$$

Final form: The best $\hat{\theta}$ is

$$\arg\min_{\theta} -\frac{1}{n} \sum_{i=1}^{n} \left(y_i \phi(x_i)^T \theta + \log\left(\sigma\left(-\phi(x_i)^T \theta\right)\right)\right)$$

CC BY NC SA

Gradient of Average Cross-Entropy Loss

- Want to minimize

$$\mathbf{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}(y_i\phi(x_i)^T\theta + \log(\sigma(-\phi(x_i)^T\theta)))$$

- Take Derivative:

$$\nabla_\theta \mathbf{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\nabla_\theta y_i\phi(x_i)^T\theta + \nabla_\theta \log\left(\sigma\left(-\phi(x_i)^T\theta\right)\right)$$

$$= -\frac{1}{n}\sum_{i=1}^{n} y_i\phi(x_i) + \nabla_\theta \log\left(\sigma\left(-\phi(x_i)^T\theta\right)\right)$$

$$\nabla_\theta \mathbf{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} y_i\phi(x_i) + \frac{1}{\sigma\left(-\phi(x_i)^T\theta\right)}\boxed{\nabla_\theta\sigma\left(-\phi(x_i)^T\theta\right)}$$

Derivative

$$\frac{d}{dt}\sigma(t) = \sigma(t)\sigma(-t)$$

$$= -\frac{1}{n}\sum_{i=1}^{n} y_i\phi(x_i) + \frac{\sigma\left(-\phi(x_i)^T\theta\right)}{\sigma\left(-\phi(x_i)^T\theta\right)}\sigma\left(\phi(x_i)^T\theta\right)\nabla_\theta(-\phi(x_i)^T\theta)$$

$$= -\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \sigma\left(\phi(x_i)^T\theta\right)\right)\phi(x_i)$$

CC BY NC SA

Gradient Descent Algorithms

Slides from Spring 2020.

- Set derivative = 0 and solve for $\hat{\theta}$
  - No general analytic solution
  - Solved using numeric methods

Gradient Descent

$\theta^{(0)} \leftarrow$ initial vector (random, zeros ...)

For τ from 0 to convergence:

$$\theta^{(\tau+1)} \leftarrow \theta^{(\tau)} - \rho(\tau) \left(\frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \mathbf{L}_i(\theta) \Big|_{\theta=\theta^{(\tau)}} \right)$$

Stochastic Gradient Descent

$\theta^{(0)} \leftarrow$ initial vector (random, zeros ...)

For τ from 0 to convergence:

$\mathcal{B} \sim$ Random subset of indices

$$\theta^{(\tau+1)} \leftarrow \theta^{(\tau)} - \rho(\tau) \left(\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \mathbf{L}_i(\theta) \Big|_{\theta=\theta^{(\tau)}} \right)$$

Very Similar Algorithms

Assuming Decomposable Loss Functions

54

[Extra] AUC of the Random Predictor

- Decision Boundaries
- Linear Separability
- Performance Metrics
- Adjusting the Classification Threshold

BY NC SA

[Extra] What is the "worst" AUC and why is it 0.5?

$$\text{TPR} = \frac{TP}{TP + FN}$$
$$\text{FPR} = \frac{FP}{FP + TN}$$

Best possible AUC = 1. Terrible AUC = 0.5.

- Random predictors have an AUC of around 0.5. Why?

A **random predictor** randomly predicts P(Y = 1 | x) to be uniformly between 0 and 1.

On average, if your dataset is size n_1+n_0 (with n_1 true class 1's and n_0 true class 0's):

If T = 0.5:

	0	1
0	TN = 0.5 n_0	FP = 0.5 n_0
1	FN = 0.5 n_1	TP = 0.5 n_1

FPR = 0.5 n_0/((0.5 + 0.5)n_0) = 0.5

TPR = 0.5 n_1/((0.5 + 0.5)n_1) = 0.5

Point on ROC curve is (0.5, 0.5).

If T = 0.8:

	0	1
0	TN = 0.8 n_0	FP = 0.2 n_0
1	FN = 0.8 n_1	TP = 0.2 n_1

FPR = 0.2 n_0/((0.2 + 0.8)n_0) = 0.2

TPR = 0.2 n_1/((0.2 + 0.8)n_1) = 0.2

Point on ROC curve is (0.2, 0.2).

If T = 0.3:

	0	1
0	TN = 0.3 n_0	FP = 0.7 n_0
1	FN = 0.3 n_1	TP = 0.7 n_1

FPR = 0.7 n_0/((0.7 + 0.3)n_0) = 0.7

TPR = 0.7 n_1/((0.7 + 0.3)n_1) = 0.7

Point on ROC curve is (0.7, 0.7).

CC BY NC SA

[Extra] What is the "worst" AUC and why is it 0.5?

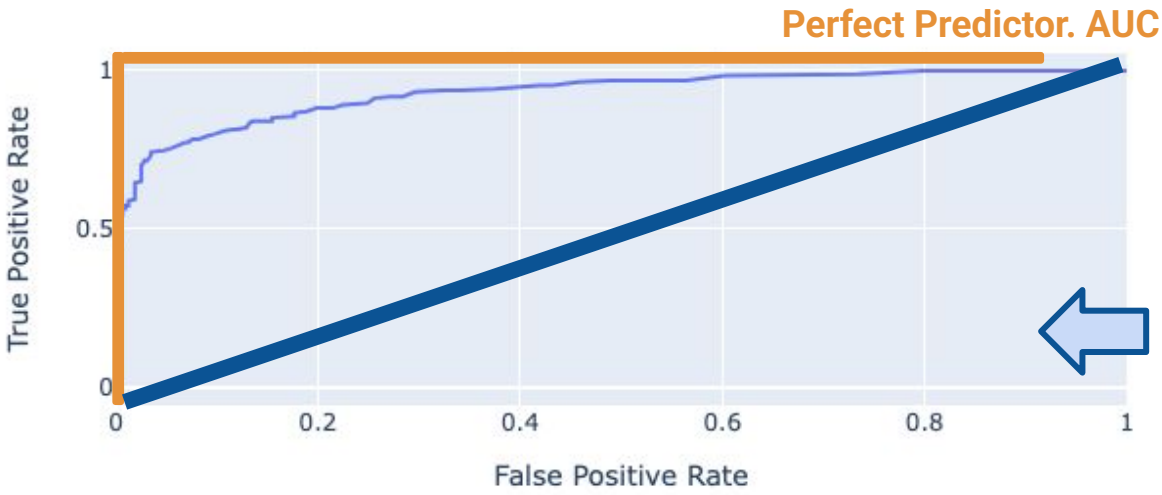
$$\text{TPR} = \frac{TP}{TP + FN}$$

$$\text{FPR} = \frac{FP}{FP + TN}$$

Best possible AUC = 1. Terrible AUC = 0.5.

- Random predictors have an AUC of around 0.5. Why?

A **random predictor** randomly predicts P(Y = 1 | x) to be uniformly between 0 and 1.



Random Predictor. AUC = 0.5.

Area Under Curve (AUC) of random predictor is the area of this triangle.

CC BY NC SA